

SPECTRE

Local-First OSINT & Security Investigation Platform

WINDOWS EDITION

Evidence-driven investigation, passive intelligence collection, graph analysis, deterministic findings, reporting, and tightly controlled authorized verification.

Platform	Backend	Database	Frontend
Windows 10/11	Python / FastAPI	MySQL	HTML / CSS / JavaScript

Created by George Youkhanna AKA DodogeUser

1. Overview

SPECTRE is a local-first cybersecurity investigation and OSINT platform built around evidence integrity, provenance, analyst traceability and explainable workflows. It combines case management, passive public intelligence, immutable evidence records, graph/timeline analysis, deterministic findings, reporting, and optional controlled active verification for explicitly authorized targets.

Core principles

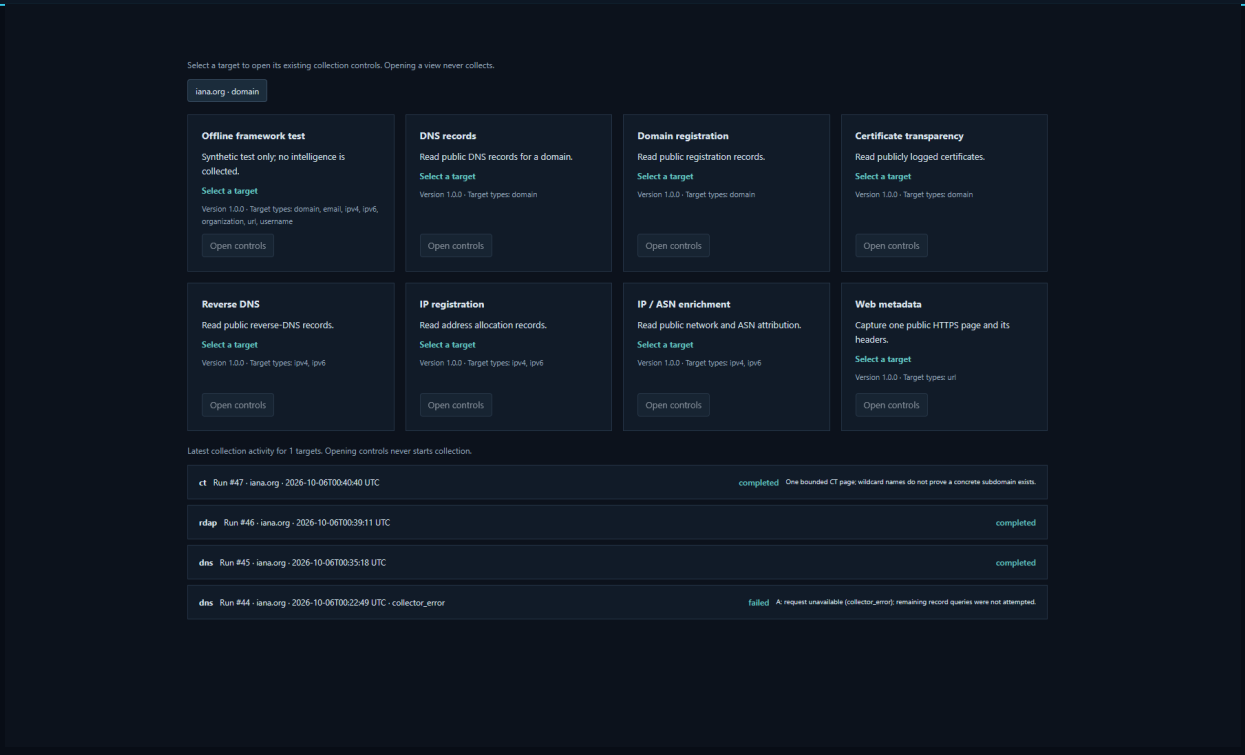
- Evidence first: observations retain source, timestamp, hash, collector identity/version and case context.
- Local first: the application and investigation database run on the analyst's Windows environment.
- Explainable: findings remain tied to supporting evidence.
- Safe by default: active assessment is separately gated and disabled by default.
- Exact scope: discovered assets never automatically inherit active authorization.

Technology stack

Layer	Implementation
Backend	Python / FastAPI
Database	MySQL + SQLAlchemy + Alembic
Frontend	HTML / CSS / vanilla JavaScript
Graph	Cytoscape.js
Reports	HTML and PDF exports
Windows launch	SPECTRE.exe convenience launcher / supported startup mechanism

2. SPECTRE Interface

The following images are real screenshots from the SPECTRE application supplied by the project creator.

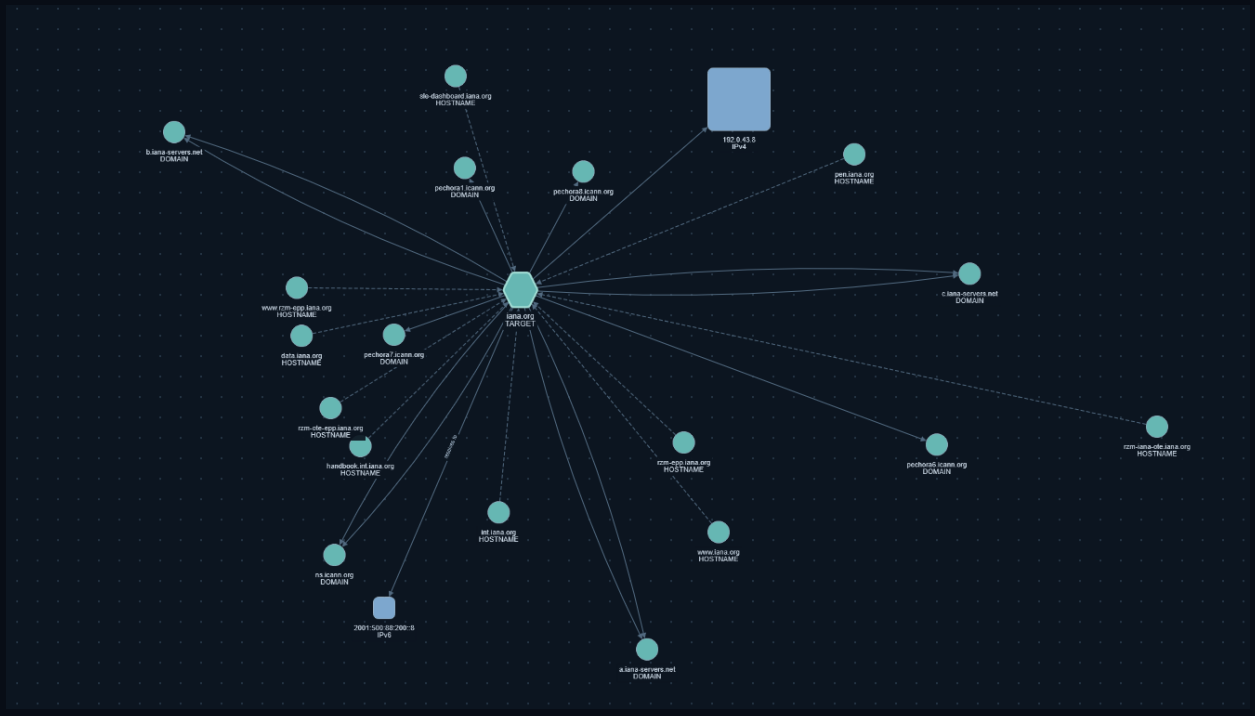


Collectors workspace. Passive modules include DNS records, domain registration, Certificate Transparency, reverse DNS, IP registration, IP/ASN enrichment and web metadata. The activity area records completed and failed collection runs.

Collectors are target-aware. Opening a collector view does not itself perform collection; the analyst selects the target and controls the action from the application.

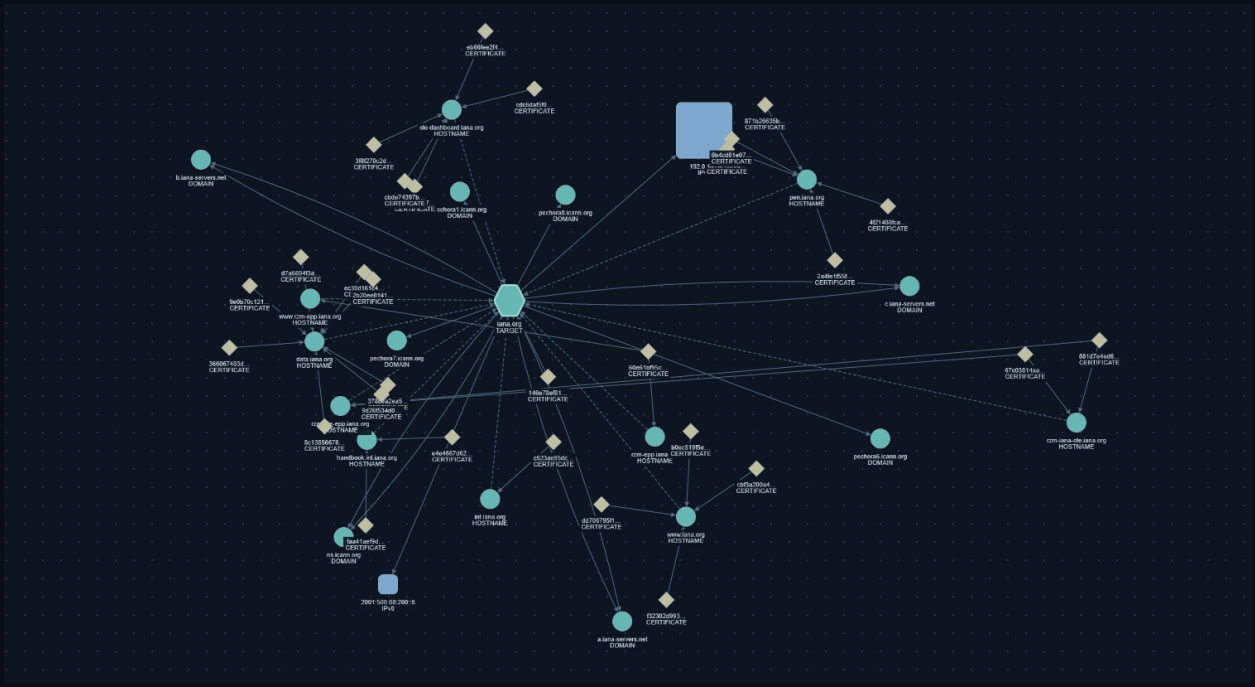
3. Investigation Graph

SPECTRE's graph turns evidence-backed entities and relationships into an interactive investigation map. The central target can be connected to hostnames, domains, IP addresses, certificates and other discovered infrastructure.



Focused graph view. A target-centered investigation showing domain/hostname and IP relationships while keeping the graph readable.

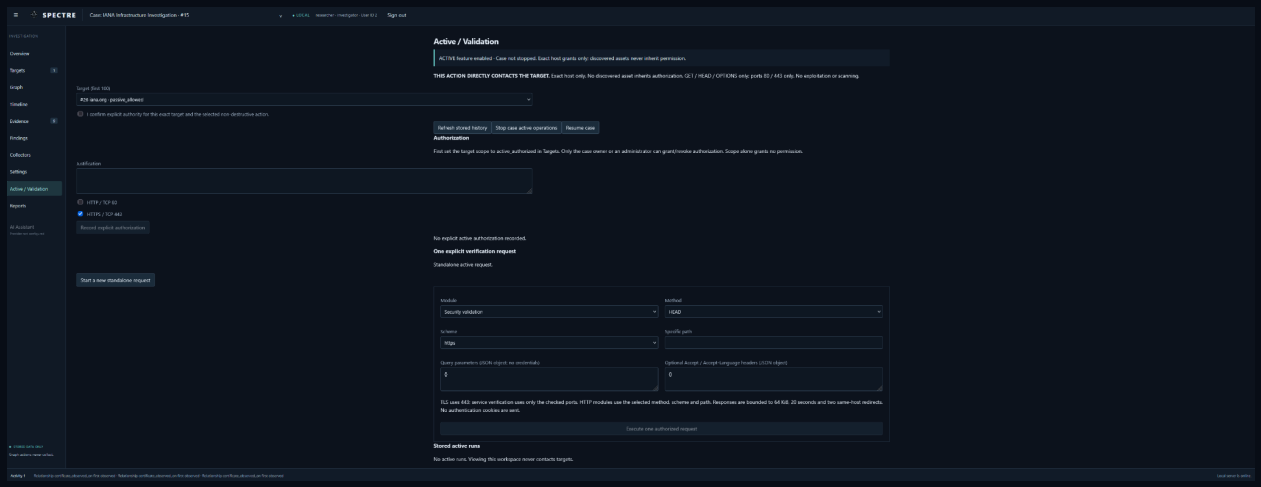
Expanded certificate relationships



Expanded graph view. Certificate nodes are revealed alongside infrastructure entities, allowing the analyst to inspect a denser relationship model without losing evidence context.

- Organic, Hierarchical, Radial and Grid layouts.
- Draggable nodes with browser-local position, size, pan and zoom persistence.
- Filters, inspector controls and certificate/hostname collapsing.
- Curved parallel edges and visual differentiation between entity types.
- Graph relationships remain grounded in collected case evidence.

4. Authorized Active / Validation



Active / Validation workspace. SPECTRE visibly separates active verification from passive investigation and requires explicit authorization before a request can contact a target.

SPECTRE includes bounded, non-destructive verification for systems the analyst is explicitly authorized to assess. The feature is disabled by default in shipped configuration and uses two independent safety gates.

Gate	Purpose
Global active feature enablement	Makes the feature available; it does not authorize any target.
Exact-target authorization	Records explicit permission for the exact target and selected non-destructive scope.

Supported verification

- HTTP/HTTPS verification with safe methods such as GET, HEAD and OPTIONS.
- TLS presented-certificate inspection.
- Tightly bounded web service verification on ports 80 and 443.
- Security-header and safe endpoint validation.
- Constrained manual requests and Finding -> Verify workflows.
- Verification states including OBSERVED, VERIFIED, NOT CONFIRMED and FAILED TO VERIFY.

SPECTRE v1.0 excludes exploit execution, automated SQLi/XSS exploitation, credential attacks, brute force, unrestricted scanning, malware, persistence, destructive requests, denial-of-service, stealth/evasion and automated attack chaining.

5. Evidence, Findings & Reporting

• Evidence Explorer: review collected observations and their provenance.

• Timeline: reconstruct investigation activity chronologically.

• Findings: deterministic conclusions tied to supporting evidence.

• Historical integrity: new verification evidence is added rather than rewriting the original observation.

• Reports: deterministic HTML/PDF exports preserve evidence traceability and SPECTRE branding.

• AI interface: evidence-grounded and read-only; AI analysis is not evidence and cannot freely browse, run commands or launch collectors.

Evidence vs. analysis

SPECTRE deliberately distinguishes collected evidence from derived interpretation. This lets another analyst reconstruct why a finding exists, which observations supported it, and what later verification did or did not confirm.

6. Windows Installation

This document covers the Windows edition only. Use the official SPECTRE release package and the instructions shipped with that release. Never copy another analyst's live .env file, virtual environment, database or secrets.

Requirement	Notes
Operating system	Windows 10 or Windows 11
Python	Use the supported version documented by the release
Database	MySQL with a dedicated SPECTRE database/user
Browser	Modern Chromium/Firefox-class browser

Recommended flow

• 1. Download or clone the official SPECTRE release into a clean Windows folder.

• 2. Run the documented Windows setup procedure to create the Python environment and install dependencies.

• 3. Create local configuration from the provided environment example and set your own database credentials/secrets.

• 4. Create the MySQL database and least-privilege SPECTRE database user.

• 5. Apply the included Alembic migrations.

• 6. Bootstrap the first application account using the supported procedure.

• 7. Run the included health/doctor check if present.

• 8. Start SPECTRE with the Windows launcher or documented startup command.

• 9. Open the local SPECTRE address and sign in.

After setup, the SPECTRE.exe launcher provides a convenient Windows startup path. It is a launcher for the existing application, not a separate implementation of SPECTRE.

7. Typical Investigation Workflow

Step	Action
1	Create a case and define its purpose.
2	Add the intended target and confirm scope.
3	Run permitted passive collectors.
4	Review evidence, provenance, timestamps and hashes.
5	Explore graph, timeline and deterministic findings.
6	Review supporting evidence before accepting conclusions.
7	If explicitly authorized, use bounded active verification after both safety gates are satisfied.
8	Export the case report in HTML/PDF.

Exact-target authorization

Authorizing `example.com` does not automatically authorize `api.example.com`, a discovered IP address, or third-party/CDN infrastructure. Each active target must independently satisfy SPECTRE's authorization controls.

8. Security Model

Control	Purpose
Evidence provenance	Source, time, collector identity/version and case context.
Evidence hashing	Integrity checking and traceability.
Historical immutability	Preserves earlier evidence instead of silently rewriting it.
RBAC	Restricts sensitive actions by role.
Audit trail	Records security-relevant and investigation actions.
Authorization grants	Separates discovery from permission.
Active run controls	Cancellation and case-level safety controls.
Bounded networking	Timeouts, response limits, redirect limits and rate/concurrency controls.

Local security

Protect the Windows account, MySQL credentials, environment configuration, investigation database, backups and exported reports. Do not commit live secrets or a production .env file to a public repository.

9. Project Structure - High Level

Area	Purpose
app/	FastAPI application, routes, services, models and application logic.
app/static/	Frontend assets and SPECTRE branding.
alembic/	Database migration history.
tests/	Backend regression and safety tests.
docs/	Installation, handoff, release and operational documentation.
scripts/	Supported setup, maintenance and platform helper scripts.
output/	Generated output such as reports when applicable.
dist/	Release-candidate / packaged distribution artifacts when built.

Windows launcher

The launcher is intended to locate the installation, verify runtime files, avoid unnecessary duplicate instances, start the local application safely, wait for readiness and open the browser. It should not require administrator privileges, modify firewall rules, embed secrets or bypass SPECTRE's controls.

10. Responsible Use & Troubleshooting

SPECTRE is intended for lawful cybersecurity investigation, education, defensive analysis and authorized security validation. Passive public information does not imply permission to actively test a system. Active verification should only be used on systems the analyst owns or is clearly authorized to assess, within the approved scope.

- If the browser cannot connect, confirm the SPECTRE process/launcher is still running.
- If startup fails, read the launcher/server error rather than starting repeated duplicate instances.
- If database initialization fails, verify MySQL and the configured SPECTRE database/user.
- If migrations fail, resolve the migration issue before using the application.
- If active verification is unavailable, remember that global feature enablement and exact-target authorization are separate requirements.



Created by George Youkhanna AKA DodogeUser

SPECTRE - Windows Edition